项目工作总结

边缘 AI 云台跟随系统

Jetson Orin Nano + STM32 + 总线舵机 全链路自研

项目信息	
项目名称	边缘 AI 云台跟随系统（Edge AI Gimbal）
项目周期	2026 年 10 月 17 日（核心开发 1 天，含前期设备调试）
担任角色	全栈开发：硬件选型 / 嵌入式固件 / 视觉算法 / 系统集成 / 文档输出
技术方向	边缘计算、嵌入式系统、计算机视觉、运动控制
项目性质	个人独立完成，从需求分析到代码交付
代码仓库	github.com/ZELDA-LINK-F/edge-ai-gimbal
交付物	20 个源文件 + 8 篇技术文档 + 7 页项目报告

关键词：边缘计算 · TensorRT · 目标检测 · 云台控制 · 裸机固件 · 总线协议 · 系统集成

目 录

01 一、项目概述 —— 项目背景、目标、整体架构、项目特点

02 二、技术栈详解 —— 硬件清单、软件依赖、开发工具、选型对比

03 三、系统架构与工作原理 —— 分层架构、数据流、通信协议、核心算法

04 四、详细实现 —— 硬件设计、STM32 固件、Jetson 视觉、工程工具

05 五、技术难点与解决方案 —— 8 个真实难点的分析与解决

06 六、测试与验证 —— 测试方法、性能数据、功能验证、边界测试

07 七、成果与量化数据 —— 性能指标、实测统计、交付物清单

08 八、学习收获与反思 —— 技术能力、工程方法、不足与改进

09 九、后续规划 —— 近期、中期、长期路线图

10 附录 —— 引脚分配、命令速查、参考文献

一、项目概述

1.1 项目背景

在边缘 AI 的学习路径中，绝大多数示例都停留在「模型能在开发板上跑起来、能打印出检测结果」这一阶段。这类 demo 虽然能验证模型的可用性，却没有回答一个更关键的问题：AI 的推理结果如何真正作用于物理世界？

传统的「AI + 机械」方案通常依赖高性能 PC 加独立显卡，功耗在 300W 以上，体积庞大，几乎不可能用于移动平台。而真正的嵌入式 AI 场景（机器人、无人机、智能监控、工业分拣）要求设备在十几瓦的功耗预算内完成实时感知与决策。

本项目希望验证一个完整的工程命题：在 15W 级功耗的边缘计算平台上，能否实现「实时视觉感知 → 决策计算 → 机械执行」的全链路闭环，并且性能满足实际使用要求？

为此，项目选择了一条完全自研的技术路线：不购买现成的云台套件，不调用封装好的机器人框架，而是从硬件选型、接线设计、嵌入式固件、总线协议、视觉算法到系统集成全部自己实现。这样做的目的，是在解决问题的过程中真正理解每一层的原理，而不是停留在「调库」层面。

1.2 项目目标

项目的目标分为三个层次：

层次	目标	验收标准
功能目标	摄像头检测到人后，云台自动转向目标	人员横向移动时，云台实时跟随
性能目标	检测帧率 ≥ 30 FPS，端到端延迟 < 100ms	实测 37 FPS，延迟 < 50ms
工程目标	全链路自研、可复现、可维护、可交付	开源仓库 + 完整文档 + 一键构建

1.3 系统整体架构

系统采用「感知层 → 决策层 → 执行层」的三层架构，各层职责清晰、接口明确：

层级	硬件载体	核心软件	主要职责
感知层	IMX219 CSI 摄像头	camera.py / GStreamer	图像采集，1920×1080@30fps，多

			帧预热保证曝光收敛
决策层	Jetson Orin Nano Super	YOLOv8n + TensorRT	目标检测、偏移计算、角度映射、平滑滤波
执行层	STM32F103VET6 + LX-16A	裸机固件 + servo.py	命令解析、二进制帧编码、总线协议、舵机驱动

完整数据流如下：

```
摄像头帧 (1280x720 BGR)
   │
   ├─ 180° 翻转修正（摄像头倒装）
   ▼
YOLOv8n 检测 (TensorRT FP16, 27ms)
   │
   ├─ 筛选 person 类，取面积最大者
   ▼
计算水平中心 cx → 归一化偏移 offset = (cx - 640) / 640   ∈ [-1, +1]
   │
   ├─ 映射：target = 500 + offset × (PAN_MAX - PAN_MIN) / 2
   ▼
指数平滑滤波：pan = pan + (target - pan) × 0.25
   │
   ├─ 安全限位：clamp(pan, 400, 600)
   ▼
串口文本命令 "S1,496,200\r\n"  (115200 8N1)
   │
   ▼
STM32 解析 → LX-16A 二进制帧（10 字节）
   │
   ▼
舵机转动，位置反馈 < 50ms
```

1.4 项目特点

- 全链路自研：从寄存器级固件到视觉算法，不使用现成机器人框架
- 真实硬件验证：不是仿真，而是可以跑的物理系统
- 工程化交付：自动化构建、自动化烧录、系统化文档
- 可量化：所有性能指标均有实测数据支撑

二、技术栈详解

本章详细说明项目使用的全部技术组件，包括硬件规格、软件依赖、版本号、以及选型理由。技术选型的核心原则是：在满足性能要求的前提下，优先选择功耗低、依赖少、可维护性好的方案。

2.1 硬件技术栈

2.1.1 主控平台：Jetson Orin Nano Super 8GB

项目	规格
型号	NVIDIA Jetson Orin Nano Super Developer Kit
AI 算力	67 TOPS（INT8）
GPU	1024 核 Ampere 架构 GPU，含 32 个 Tensor Core
CPU	6 核 Arm Cortex-A78AE @ 1.5GHz
内存	8GB LPDDR5（与 GPU 共享的统一内存）
存储	NVMe SSD（本项目使用 128GB）
功耗	7W / 15W / 25W 多档可调（本项目用 15W）
系统	JetPack 6.2.1（L4T 36.4.7，内核 5.15.148-tegra）
软件栈	CUDA 12.6、TensorRT 10.3、cuDNN、OpenCV、GStreamer

选型理由：

- 统一内存架构：CPU 和 GPU 共享 8GB 内存，避免数据拷贝开销，适合端到端推理
- 原生 TensorRT 支持：可从 PyTorch 模型直接导出 TensorRT 引擎，获得 1.4~3 倍加速
- 原生 CSI 接口：可直接连接 IMX219 等树莓派兼容摄像头，无需 USB 采集卡
- 功耗可控：15W 档位即可跑满 YOLOv8n 实时推理，远低于 PC + GPU 方案
- 完整的 JetPack 软件栈：CUDA、TensorRT、OpenCV、GStreamer 开箱即用

2.1.2 视觉传感器：IMX219 CSI 摄像头

项目	规格
传感器	Sony IMX219，800 万像素 CMOS
最大分辨率	3280 × 2464 @ 21fps
本项目配置	1920 × 1080 @ 30fps（检测时降到 1280 × 720）

接口	MIPI CSI-2，15pin FPC 转 22pin 排线
驱动	nvarguscamerasrc（NVIDIA GStreamer 插件）
设备树	tegra234-p3767-camera-p3768-imx219-dual.dtbo

选型理由：

- CSI 接口直连：相比 USB 摄像头，延迟更低、CPU 占用更小
- 原生 GStreamer 支持：可直接接入 nvarguscamerasrc，走硬件 ISP 通路
- 成本低、生态好：树莓派兼容模块，价格便宜、资料丰富

2.1.3 协处理器：STM32F103VET6

项目	规格
内核	Arm Cortex-M3 @ 72MHz
Flash	512 KB
SRAM	64 KB
封装	LQFP100（100 引脚）
开发板	野火「指南者」F103VET6
外设	3×USART、2×SPI、2×I2C、2×CAN、多路定时器、ADC/DAC
烧录	板载 CH340 + 一键 ISP 电路（DTR/RTS 控制 BOOT0/NRST）

选型理由：

- 实时性：裸机运行，中断响应微秒级，适合驱动总线设备
- 资源充足：512KB Flash 和 64KB RAM 足够实现协议栈
- 成本低：芯片单价十几元，适合量产
- 学习价值高：寄存器级开发能真正理解 MCU 工作原理

2.1.4 执行机构：LX-16A 总线舵机

项目	规格
类型	串行总线舵机（半双工 UART）
工作电压	6 ~ 8.4V（本项目用 7.4V 锂电池）
通信波特率	115200 bps（默认）

位置范围	0 ~ 1000，对应 0 ~ 240°（分辨率 0.24°）
堵转扭矩	约 17 kg·cm
总线能力	理论最多 253 个舵机级联，每个有独立 ID
反馈信息	当前位置、温度、输入电压、负载状态
保护机制	过压/欠压/过温自动卸载电机并 LED 报警

选型理由：

- 总线拓扑：多个舵机并联在一根信号线上，只需 3 根线（GND/VIN/SIG）即可级联
- 位置反馈：可读取真实位置，实现闭环验证
- 保护完善：内置过压/过温保护，降低烧毁风险

2.1.5 其他硬件

部件	型号/规格	作用
USB 转串口	CH340G（板载）	STM32 与 PC/Jetson 通信、ISP 烧录
电源	7.4V 锂电池组	舵机独立供电（避免拉垮主板）
连接	面包板 + 杜邦线	共地汇流、信号分线
烧录方式	一键 ISP（DTR/RTS 控制）	无需手动拨 BOOT0 跳线

2.2 软件技术栈

2.2.1 编程语言

语言	版本/标准	使用场景	代码量
C	C99（-std=gnu99）	STM32 裸机固件	约 700 行
Python	3.10（Jetson 系统自带）	Jetson 视觉、控制、工具	约 800 行
PowerShell	5.1	自动烧录脚本、构建脚本	约 150 行

2.2.2 嵌入式开发技术栈

技术	版本	说明
编译器	arm-none-eabi-gcc 14.2.1	Arm 官方 GNU 工具链，裸机交叉编译
构建	Makefile + 手动命令	无 CMake，保持依赖最少
固件框架	裸机（Bare-metal）	不使用 HAL/标准外设库/CMSIS
启动代码	手写 startup.c	自定义中断向量表 + Reset_Handler
链接脚本	手写 linker.ld	512K Flash @0x08000000，64K RAM @0x20000000
烧录工具	FlyMcu V0.188	野火推荐的 ISP 烧录工具
调试手段	串口打印 + 自定义诊断命令	无调试器，纯串口调试

为什么不用 HAL 库?

- 学习目的：直接操作寄存器才能真正理解时钟树、GPIO 模式、串口配置
- 体积控制：裸机固件仅 4.5KB，HAL 版本通常在 20KB 以上
- 可控性：不依赖 ST 的库更新，代码完全自主

2.2.3 视觉与 AI 技术栈

技术	版本	用途
OpenCV	4.x（JetPack 自带，含 GStreamer 支持）	图像解码、颜色空间转换、几何变换
GStreamer	1.0	摄像头管线构建
nvarguscamerasrc	JetPack 自带	NVIDIA CSI 摄像头 GStreamer 插

		件
Ultralytics	8.3.213	YOLOv8 模型加载与推理
YOLOv8n	6.25 MB / 3,157,200 参数	目标检测模型（COCO 80 类）
TensorRT	10.3	模型优化与推理加速（FP16）
CUDA	12.6	GPU 并行计算运行时
PyTorch	JetPack 自带	模型加载后端
NumPy	1.x	数组运算
Pillow	9.x	图像读写

2.2.4 大模型与语音技术栈（扩展能力）

技术	版本/模型	用途
Ollama	最新版	本地大模型运行时
qwen2.5vl:3b	3.2 GB	视觉大模型，图像理解（10 秒/张）
faster-whisper	CPU 模式（int8）	语音识别（加载 3.6s，推理 3.4s）

说明：视觉大模型与 YOLO 采用「常驻 + 按需」架构 —— YOLO 常驻内存负责实时感知，VLM 仅在用户主动提问时加载。这是因为 VLM 占用 5.4GB 显存，无法与摄像头采集同时运行。

2.2.5 通信技术栈

技术	版本	用途
pyserial	3.5	Python 串口通信
UART	115200 8N1	STM32 ↔ Jetson 文本协议
半双工总线	115200 8N1	STM32 ↔ LX-16A 二进制协议
SSH / SCP	OpenSSH	Jetson 远程开发与文件传输

2.3 开发工具链

类别	工具	用途
代码编辑	VS Code / 任意编辑器	源码编写
版本控制	Git 2.55.0	代码管理、GitHub 托管
远程开发	SSH + SCP	在 PC 上开发 Jetson 应用

串口调试	Python pyserial 脚本	自定义测试脚本
烧录	FlyMcu V0.188	STM32 ISP 烧录
文档	python-docx + LibreOffice	项目报告生成
渲染验证	PyMuPDF	PDF 排版检查

2.4 技术选型对比

2.4.1 主控平台对比

方案	功耗	AI 算力	成本	结论
PC + RTX 显卡	300W+	很高	高	❌ 体积功耗不适合嵌入式
树莓派 5	5~8W	低（无 NPU）	低	❌ 跑不动实时检测
ESP32	0.5W	极低	极低	❌ 无法运行视觉模型
Jetson Orin Nano	15W	67 TOPS	中	✅ 性能/功耗/成本平衡最佳

2.4.2 舵机方案对比

方案	接线	反馈	级联	结论
PWM 舵机（SG90/MG996R）	每个 1 根信号线	无	不方便	❌ 多舵机时线缆爆炸
串行总线舵机（LX-16A）	3 根线并联	有	支持 253 个	✅ 本项目选择

2.4.3 检测模型对比

模型	大小	帧率（Jetson）	精度	结论
YOLOv8n	6.25 MB	37 FPS	够用	✅ 选择
YOLOv8s	22 MB	约 15 FPS	更高	⚠ 帧率不足
YOLOv8m	52 MB	约 6 FPS	高	❌ 太慢

MobileNet-SSD	24 MB	约 20 FPS	较低	⚠ 精度不足

2.4.4 推理后端对比

后端	耗时（640 输入）	说明
PyTorch FP32	27.3 ms	基线
TensorRT FP16	19.6 ms	✅ 1.39 倍加速，选择此方案
说明	720 输入时无明显提升	瓶颈转移到图像预处理

三、系统架构与工作原理

3.1 分层架构设计

系统按照职责划分为三个独立层次，层与层之间通过明确定义的接口通信，便于单独测试和替换。

3.1.1 感知层

负责把物理世界转换为数字图像。核心挑战是「时间」与「质量」：

- 时间：CSI 摄像头从加电到输出稳定画面需要几十帧，自动曝光（AE）和自动白平衡（AWB）需要收敛
- 质量：直接抓第 1 帧会得到全黑或纯噪声图像

解决方案：预抓 90 帧丢弃，只保留最后一帧。这是本项目踩过的坑之一（详见第五章）。

3.1.2 决策层

负责把图像转换为控制指令。分为四步：

步骤	输入	处理	输出
1. 目标检测	1280×720 BGR 帧	YOLOv8n + TensorRT FP16	检测框列表（类别/置信度/坐标）
2. 目标筛选	检测框列表	筛选 person 类，取面积最大者	单个目标框
3. 偏移计算	目标框坐标	归一化到 [-1, +1]	水平偏移量
4. 角度映射	偏移量	线性映射 + 指数平滑 + 安全限位	舵机目标位置 0~1000

3.1.3 执行层

负责把数字指令转换为机械动作。分为两个子层：

- STM32 固件：接收文本命令，编码为 LX-16A 二进制帧，通过 USART2 发送
- LX-16A 舵机：解析二进制帧，驱动无刷电机转动到目标位置

3.2 通信协议设计

3.2.1 Jetson ↔ STM32：文本协议

设计原则：可读性强、便于调试、易于扩展。采用 ASCII 文本行协议，以 \r\n 结尾。

命令	格式	功能	示例
心跳	PING	检测链路是否正常	→ PONG
运动	S<id>,<pos>	舵机转到指定位置	S1,500
定时运动	S<id>,<pos>,<ms>	在 ms 毫秒内匀速转到目标	S1,500,1000
广播	S254,<pos>	所有舵机同时运动	S254,500
读位置	R<id>	读取舵机当前位置	R1
改 ID	ID<old>,<new>	修改舵机 ID（掉电保存）	ID1,2
装载	LD<id>,<0|1>	0=卸载电机 1=装载电机	LD1,1
扫描	SCAN	扫描 ID 1~20 的在线舵机	→ 列出应答的 ID
回环	LOOP	PA2 发 0x55，PA3 收，验证线路	→ 收到/未收到
波特率扫描	BAUD	遍历 9600~1000000 找舵机波特率	→ 哪个速率有应答

3.2.2 STM32 ↔ 舵机：LX-16A 二进制协议

帧格式：

```
┌──────┬──────┬────┬────────┬─────┬──────────────┬──────────┐
│ 0x55 │ 0x55 │ ID │ Length │ Cmd │ 参数...      │ Checksum │
└──────┴──────┴────┴────────┴─────┴──────────────┴──────────┘

 帧头(2)      1    1       1     N           1
```

关键规则：

- Length + 3 = 整包长度（从帧头到校验和）
- Length = ID(1) + Cmd(1) + 参数长度 + Checksum(1)
- Checksum = ~(ID + Length + Cmd + 参数...) & 0xFF

位置控制指令（Cmd=1，Length=7）：

```
55 55 01 07 01 F4 01 E8 03 16
      ↑  ↑  ↑  ↑──────┘  ↑
```

```
      ID 长令 角度 时间  校验和
         7    500   1000ms
```

3.3 核心算法

3.3.1 偏移映射算法

目标是把「人在画面中的位置」映射为「舵机的角度」。采用线性映射 + 指数平滑：

```
# 1. 计算目标水平中心
cx = (x1 + x2) / 2.0

# 2. 归一化到 [-1, +1]
offset = (cx - W/2) / (W/2)

# 3. 线性映射到舵机范围
target = PAN_CENTER + offset * (PAN_MAX - PAN_MIN) / 2

# 4. 指数平滑（避免抖动）
pan = pan + (target - pan) * SMOOTH
```

平滑系数 SMOOTH 的作用：

取值	效果	适用场景
0.10	非常平滑，有明显滞后	静态展示
0.25	平滑且响应及时（本项目采用）	通用跟随
0.50	灵敏，可能有轻微抖动	快速移动目标
1.00	每帧直接到位，可能产生异响	不推荐

3.3.2 安全限位算法

摄像头使用 FPC 硬排线，反复弯折会断裂。因此在软件层强制限制舵机运动范围：

```
PAN_MIN, PAN_MAX = 400, 600     # ±100 单位 = ±24°

def clamp(self, sid, pos, bypass=False):
    lo, hi = LIMITS.get(sid, (0, 1000))
    if pos < lo:  return lo, True
    if pos > hi:  return hi, True
    return pos, False
```

保留 bypass 参数用于特殊场景（如调试时临时放开），但正常运行时强制限制。

3.3.3 无电阻总线驱动算法

LX-16A 是半双工总线，多个设备共用一根信号线。传统方案需要串接 1kΩ 电阻防止总线冲突。本项目通过 GPIO 模式动态切换实现无电阻方案：

```
发送前：PA2 切换为复用推挽输出（强驱动，不需要上拉）
    ↓
发送完整帧
    ↓
等待 TC 标志（最后一个字节完全移出移位寄存器）
    ↓
PA2 切换为浮空输入（高阻态，让出总线）
    ↓
此时舵机可以安全地在总线上应答，不会产生冲突
```

PA3 同时配置为输入上拉，为总线提供空闲高电平。

四、详细实现

4.1 硬件设计与接线

4.1.1 供电系统

系统有两套独立电源：

电源	供给对象	电压	说明
USB 5V	Jetson、STM32	5V	来自 USB 线，电流有限
锂电池	LX-16A 舵机	7.4V	舵机堵转电流可达 2~3A

为什么舵机必须独立供电？

- 舵机启动/堵转瞬间电流可达 2~3A，远超 USB 的 500mA~1A 供电能力
- 如果从主板取电，会导致主板电压跌落、重启，甚至损坏 USB 口
- 独立供电后，舵机的电流冲击不会影响 Jetson 和 STM32

4.1.2 共地方案

UART 信号电平是相对于各自的地判断的。两套电源如果不共地，就没有共同的「零点」，接收方无法正确识别高低电平，通信必然失败。

实现方式：用面包板把以下三点接到同一条负极轨：

```
面包板负极轨
  ├── 电池黑线（负极）
  ├── 舵机黑线（GND）
  └── 杜邦线 → STM32 板 GND（JP4 第 5 行靠板子边缘）
```

4.1.3 信号线连接

舵机线	接到	板子位置	说明
白线（SIG）	PA2	JP5 第 3 行靠中间	USART2_TX，发送命令
白线（SIG）	PA3	JP5 第 4 行靠中间	USART2_RX，接收应答 + 提供上拉

注意：PA2 和 PA3 必须接在面包板的【同一横排】。面包板的一排 5 个孔是连通的，不同排不连通。这是本项目踩过的坑之一 —— 曾经因为插在不同排，导致舵机完全收不到信号。

4.1.4 引脚分配表

引脚	功能	方向	连接
PA2	USART2_TX	输出	舵机总线（推挽发送 / 高阻让出）
PA3	USART2_RX	输入	舵机总线（上拉 + 接收应答）
PA9	USART1_TX	输出	CH340 → PC/Jetson
PA10	USART1_RX	输入	CH340 ← PC/Jetson
GND	地	-	面包板负极轨（共地）

4.2 STM32 固件实现

4.2.1 工程结构

文件	行数	职责
startup.c	约 60 行	中断向量表、Reset_Handler、默认异常处理
linker.ld	约 40 行	内存布局：Flash/RAM 起始地址、段定义
main.c	约 600 行	时钟、GPIO、串口、SysTick、命令解析、舵机协议
Makefile	约 20 行	编译规则
flash.ps1	约 150 行	FlyMcu 自动烧录（Win32 API 自动化）

4.2.2 启动流程

STM32 上电后的执行顺序：

```
上电复位
   ↓
从 0x08000000 读取初始栈指针（MSP）
从 0x08000004 读取 Reset_Handler 地址
   ↓
Reset_Handler:
```

```
   ├─ 把 .data 段从 Flash 拷贝到 RAM
   ├─ 把 .bss 段清零
   └─ 跳转到 main()
   ↓
main():
   ├─ clock_init()      配置 HSE + PLL → 72MHz
   ├─ gpio_usart_init() 配置 GPIO 模式 + 两个串口
   ├─ systick_init()    1ms 系统时钟
   └─ 主循环
```

4.2.3 时钟系统

STM32F103 最高主频 72MHz，需要外部晶振配合 PLL：

```
HSE 8MHz（外部晶振）
   ↓ ×9（PLLMULL9）
72MHz 系统时钟（SYSCLK）
   ├─ AHB  = 72MHz（HCLK）
   ├─ APB2 = 72MHz（USART1、GPIO）
   └─ APB1 = 36MHz（USART2，必须 ≤36MHz）
```

因为 USART1 和 USART2 挂在不同总线，所以波特率寄存器的值不同：

```
USART1->BRR = 72,000,000 / 115200 = 625
USART2->BRR = 36,000,000 / 115200 = 312.5 ≈ 312
```

4.2.4 命令解析器

采用非阻塞轮询 + 行缓冲，避免丢字符：

```
char line[80];
int li = 0;
char c;

while (1) {
    /* 尽快排空接收缓冲 */
    while (u1_getc(&c)) {
        if (c == '\r' || c == '\n') {
            if (li > 0) {
                line[li] = 0;
                handle_command(line);   /* 处理完整一行 */
                li = 0;
            }
        } else if (li < sizeof(line) - 1) {
            line[li++] = c;
        }
```

```
    }

    /* 每秒心跳 */
    if (millis() - last_beat >= 1000) { ... }
}
```

重要教训：第一版固件在主循环里用了 delay_ms(1)，导致每毫秒只能读 1 个字符。而 115200 波特率下每秒传 11520 字节，约每 87 微秒来一个字节 —— 严重丢字符。改成非阻塞轮询后问题解决。

4.2.5 舵机帧编码

```
static void servo_send_frame(uint8_t id, uint8_t cmd,
                             const uint8_t *params, uint8_t plen)
{
    uint8_t len = plen + 3;          /* ID + CMD + 参数 + 校验和 */
    uint16_t sum = id + len + cmd;
    for (uint8_t i = 0; i < plen; i++) sum += params[i];

    pa2_drive(1);                    /* 切推挽输出 */

    u2_putc(0x55); u2_putc(0x55);    /* 帧头 */
    u2_putc(id);
    u2_putc(len);
    u2_putc(cmd);
    for (uint8_t i = 0; i < plen; i++) u2_putc(params[i]);
    u2_putc((~sum) & 0xFF);          /* 校验和 */

    while (!(USART2->SR & USART_SR_TC)) { }   /* 等移出完成 */
    pa2_drive(0);                    /* 切高阻，让出总线 */
}
```

4.2.6 诊断命令设计

为了在没有示波器、没有逻辑分析仪的条件下调试，固件内置了三个诊断命令：

命令	原理	能定位什么问题
LOOP	PA2 发 0x55，PA3 接收（两者接在同一根线上）	信号线是否连通、面包板是否同排、引脚是否插错
SCAN	遍历 ID 1~20 发送读位置指令	舵机的真实 ID 是多少
BAUD	遍历 9600~1000000 波特率发送读指令	舵机被配置成了什么波特率

这三个命令把原本需要示波器的排查，变成了几秒钟的串口交互。

4.3 Jetson 视觉实现

4.3.1 摄像头管线

使用 GStreamer 构建零拷贝管线：

```
nvarguscamerasrc sensor-id=0
  ! video/x-raw(memory:NVMM),width=1280,height=720,framerate=30/1,format=NV12
  ! nvvidconv
  ! video/x-raw,format=BGRx
  ! videoconvert
  ! video/x-raw,format=BGR
  ! appsink drop=1 max-buffers=2
```

关键点：

- memory:NVMM：数据留在 GPU 内存，避免拷贝到 CPU
- drop=1：缓冲区满时丢弃旧帧，保证低延迟
- max-buffers=2：只保留 2 帧，避免积压

4.3.2 模型部署流程

```
YOLOv8n.pt (PyTorch 权重, 6.25MB)
   ↓ ultralytics export
YOLOv8n.onnx
   ↓ trtexec --fp16
YOLOv8n.engine (TensorRT 引擎, 8.82MB)
```

TensorRT 引擎的构建耗时约 462 秒（一次），之后加载只需几秒。注意：trtexec 导出的引擎不含类别名元数据，加载后类别名会变成 class0、class5 等，需要手动映射。

4.3.3 跟随主循环

```
class Follower:
    def step(self):
        ok, frame = self.cap.read()
        if not ok: return

        if self.flip:
            frame = cv2.rotate(frame, cv2.ROTATE_180)   # 180° 翻转修正

        dets, dt = detect_frame(self.model, frame, conf=0.4)
        person = self.biggest_person(dets)

        if person:
            x1, y1, x2, y2 = person["box"]
            cx = (x1 + x2) / 2.0
            offset = max(-1.0, min(1.0, (cx - W/2) / (W/2)))
```

```
            target = PAN_CENTER + offset * (PAN_MAX - PAN_MIN) / 2.0
            self.pan += (target - self.pan) * SMOOTH

            # 快速发送，不等应答
            p, _ = self.bus.clamp(PAN_ID, self.pan)
            self.bus.s.write(("S1,%d,200\r\n" % p).encode())
        else:
            # 没人 → 慢慢回中
            self.pan += (PAN_CENTER - self.pan) * 0.1
            self.bus.s.write(("S1,%d,400\r\n" % int(self.pan)).encode())
```

为什么用「快速发送」而不是 bus.move()?

- bus.move() 会等待舵机应答，约 400ms，太慢，跟不动目标
- 跟随循环里直接写串口，不等应答，每帧都能发新指令
- 同时每 2 秒清一次接收缓冲，避免固件的 TX 回显和 HELLO 心跳堆积

4.4 工程工具实现

4.4.1 FlyMcu 自动烧录

问题：FlyMcu 是 Delphi 编写的 GUI 程序，每次烧录都要手动点击，非常耗时。我尝试了多种自动化方案：

方案	结果	原因
UI Automation 查找控件	❌	Delphi VCL 不暴露子控件给 UIA
SendKeys 发送快捷键	❌	窗口未获得焦点，按键丢失
BM_CLICK 消息	❌	Delphi 自绘按钮不响应标准按钮消息
真实鼠标点击（窗口未置顶）	❌	窗口被遮挡，点击落在其他窗口
真实鼠标点击（窗口置顶）	✅	成功

最终实现：

```
1. 修改 FlyMcuConfig.ini 预置参数
   Port=2（对应 COM4）、PortBps=4（115200）、FileName=xxx.hex
   cbSTMWriteOption=0（不写选项字节）、cbSTMExecAfterFlash=1（烧完运行）

2. 启动 FlyMcu.exe

3. 用 Win32 API 把窗口移到 (30,30) 并置顶
```

```
   SetWindowPos(hwnd, HWND_TOPMOST, 30, 30, 720, 560, SWP_SHOWWINDOW)
   SetForegroundWindow(hwnd)

4. 枚举子窗口，按类名（TButton）+ 文本（含 "(P)"）找到「开始编程」按钮

5. 获取按钮屏幕坐标，移动鼠标 + 发送真实鼠标事件
   SetCursorPos(cx, cy)
   mouse_event(MOUSEEVENTF_LEFTDOWN, ...)
   mouse_event(MOUSEEVENTF_LEFTUP, ...)

6. 轮询 ProgramLogs.txt，检测到 "Program Success" 即成功
```

效果：一条命令完成烧录，当天迭代十余次固件，节省大量时间。

4.4.2 版本管理与文档

使用 Git 管理代码，推送到 GitHub。文档使用 python-docx 生成，LibreOffice 转换为 PDF，PyMuPDF 渲染检查排版。

五、技术难点与解决方案

本章记录项目开发中遇到的 8 个真实难点。每个难点都包含：现象描述、排查过程、根本原因、解决方案、经验教训。

1. LX-16A 协议帧格式错误（花费时间最长）

现象

- 舵机 LED 常亮，说明供电正常
- STM32 收到命令并回显 OK，说明串口通信正常
- 但舵机完全不转、也不回应读位置指令
- 尝试了 20 多种组合：不同波特率、不同时序、不同协议顺序、解除写保护、按页擦除……全部无效

排查过程

- 确认 STM32 与 PC 通信正常（PING → PONG）
- 确认信号线连通（用 LOOP 回环自测，PA2 发 0x55，PA3 能收到）
- 确认舵机有电（LED 亮）
- 下载官方协议文档，逐字节对比发送的帧
- 发现：我发的帧比官方例子多 2 个字节

根本原因

- 凭记忆编写协议，在位置控制指令里多加了「速度」2 个字节
- 导致 Length 字段与参数实际长度不匹配
- 舵机固件校验不通过，静默丢弃整个数据包（不返回任何错误）

解决方案

- 按官方文档修正：位置控制指令只有 4 字节参数（角度 2 + 时间 2）
- Length = ID(1) + Cmd(1) + 参数(4) + 校验和(1) = 7
- 修正后帧格式与官方例子完全一致：55 55 01 07 01 F4 01 E8 03 16
- 舵机立即恢复正常

经验教训

- 协议文档必须逐字节核对，不能凭记忆实现
- 写完协议编码后，先用官方例子做单元测试比对
- 嵌入式设备的协议错误通常是「静默失败」，不会有任何提示

2. Jetson 系统更新后摄像头完全消失

现象

- 之前摄像头工作正常
- 执行了系统批量更新（894 个包）
- 重启后 /dev/video0 不存在，i2cdetect 看不到任何设备

排查过程

- 检查 dmesg，没有摄像头驱动加载记录
- 检查设备树 overlay 配置
- 发现 /boot/extlinux/extlinux.conf 的 DEFAULT 被改回 primary

根本原因

- 系统更新重置了引导配置
- DEFAULT 从 JetsonIO 变成 primary
- 导致 tegra234-p3767-camera-p3768-imx219-dual.dtbo 这个 overlay 没有被加载
- IMX219 摄像头依赖这个 overlay 才能被识别

解决方案

- 执行 sudo sed -i 's/^DEFAULT primary$/DEFAULT JetsonIO/' /boot/extlinux/extlinux.conf
- 重启后摄像头恢复
- 建立习惯：Jetson 每次系统更新后都检查这个文件

经验教训

- 嵌入式设备的系统更新经常重置自定义配置
- 出问题先检查配置文件有没有被覆盖，再怀疑硬件

3. 舵机总线无信号（面包板接线问题）

现象

- 舵机有电（LED 亮）
- STM32 正常发送命令
- 舵机完全没有反应，读位置也无应答

排查过程

- 发送 LOOP 命令：PA2 发 0x55，PA3 接收
- 结果：收不到任何数据

根本原因

- PA2 和 PA3 的杜邦线插在面包板的不同横排
- 面包板的一排 5 个孔是连通的，但不同排之间不连通
- 因此 PA2 和 PA3 实际上没有连接在同一根信号线上

解决方案

- 把两根线插到同一横排
- 再次 LOOP 测试：收到 0x55，接线正常
- 舵机恢复正常通信

经验教训

- 面包板的「排」概念必须搞清楚
- 用「发什么收什么」的回环自测，能 5 秒定位线路问题
- 这是比万用表更快的排查方法

4. FlyMcu 无法自动化（GUI 程序）

现象

- 需要频繁烧录固件，手动点击 FlyMcu 很耗时
- 尝试用 UI Automation 查找按钮，返回 0 个控件
- 尝试 SendKeys 发送快捷键，无反应
- 尝试发送 BM_CLICK 消息，无反应
- 尝试真实鼠标点击，还是无反应

排查过程

- UI Automation：Delphi VCL 不暴露子控件
- SendKeys：窗口没有获得焦点
- BM_CLICK：Delphi 自绘按钮不响应标准按钮消息
- 真实鼠标点击：截图后发现 FlyMcu 窗口被其他窗口完全遮挡

根本原因

- GUI 自动化失败的原因分两类：控件不可访问、窗口不在前台
- 本次是后者 —— 点击坐标落在了遮挡窗口上

解决方案

- 用 Win32 API 枚举子窗口，按类名 TButton + 文本找到按钮

- 用 SetWindowPos 把窗口移到 (30,30) 并置顶
- 用 SetForegroundWindow 激活窗口
- 用 SetCursorPos + mouse_event 发送真实鼠标事件
- 用 ProgramLogs.txt 的追加内容判断烧录结果

经验教训

- GUI 自动化失败时，先截图看屏幕实际状态，不要凭空推测
- Delphi/VCL 程序的按钮必须用真实鼠标事件点击

5. 摄像头 FPC 排线易损（机械约束）

现象

- 摄像头使用 15pin 转 22pin 的硬质 FPC 排线
- 云台旋转时排线会被反复弯折
- 担心排线内部铜箔疲劳断裂（断裂后无法修复）

排查过程

- 分析排线的受力方式：弯折 vs 扭转
- 测量旋转角度与排线形变的关系

根本原因

- FPC 排线耐扭转能力远高于耐弯折能力
- 传统全角度云台（±120°）会严重弯折排线

解决方案

- 软件层：限制舵机运动范围到 ±24°（400~600）
- 结构层：排线走线尽量靠近转轴，使旋转时以「扭转」代替「弯折」
- 固定层：在舵机本体上用胶带固定排线根部，让应力集中在固定点
- 预留服务环：留一段松弛的弯，提供形变余量

经验教训

- 硬件设计要考虑软件约束（软件限位是硬件保护的一部分）
- 排线走线的设计原则：靠近转轴、固定根部、预留余量

6. 舵机突然全部失去响应

现象

- 连续测试 2 分钟后，两个舵机同时不响应

- STM32 仍能正常收发命令
- 舵机 LED 状态异常

排查过程

- 检查 STM32 通信（PING 正常）
- 检查信号线（LOOP 正常）
- 检查电源（电池电量不足）

根本原因

- 7.4V 锂电池耗尽
- 舵机连续转动耗电很大，2~3 分钟就可能耗尽
- 电池耗尽时舵机不会报错，只是静默不响应

解决方案

- 充电后恢复正常
- 建立电池电量检查习惯

经验教训

- 舵机测试要留意电池电量
- 电池耗尽的表现和通信故障非常相似，容易误判

7. 舵机位置读数出现 65534

现象

- 舵机转到位置 0 时，读位置返回 65534
- 其他位置都正常

排查过程

- 检查协议：位置是 16 位无符号数
- 65534 = 0xFFFE

根本原因

- 舵机顶到机械限位时有 -2 的负过冲
- 用无符号 16 位解析就变成了 65534
- 实际值应该是有符号的 -2

解决方案

- 解析时判断有符号数：v = v - 65536 if v > 32000 else v

- 显示时按实际值处理

经验教训

- 边界情况必须单独测试
- 16 位数据要明确是有符号还是无符号

8. PowerShell 远程命令引号丢失

现象

- 执行 ssh nvidia@... "python3 -c 'import servo; ...'"
- 远端报错：bash: -c: 行 1: 未预期的记号 "(" 附近有语法错误

排查过程

- 怀疑是引号转义问题
- 尝试多种转义方式，都不稳定

根本原因

- PowerShell 会处理引号、$ 变量、换行等特殊字符
- 导致传给 SSH 的命令被破坏

解决方案

- 把脚本写成文件，用 scp 上传后执行
- 避免复杂的远程内联命令

经验教训

- 复杂的远程命令不要内联，写成文件传输
- 这是跨平台开发的常见陷阱

六、测试与验证

6.1 测试方法

测试类型	方法	工具
单元测试	固件诊断命令（PING/LOOP/SCAN/BAUD）	串口脚本
功能测试	逐条验证命令（运动/读位置/改ID/广播）	Python 脚本
性能测试	连续采集 2 分钟，统计帧率与延迟	follow.py 日志
精度测试	指令位置 vs 实际位置对比	read_pos 读取
边界测试	位置 0 / 1000 / 超限 / 断电恢复	手动 + 脚本
稳定性测试	连续运行 3500+ 帧	长时间录制

6.2 性能测试数据

指标	数值	测试条件
检测帧率	37 FPS（27ms/帧）	1280×720 输入，YOLOv8n TensorRT FP16
检测准确率	100%（3509/3509 帧）	连续 2 分钟，单人目标
舵机定位精度	±1 / 1000（±0.24°）	指令值与读数对比
端到端延迟	< 50ms	从图像采集到机械动作
串口往返延迟	< 10ms	115200 波特率，8 字节命令
固件启动时间	< 100ms	上电到打印 READY
固件体积	4.5 KB	arm-none-eabi-gcc -O2
内存占用	< 2 KB RAM	含栈和全局变量

6.3 功能验证

功能	验证方法	结果
串口通信	发送 PING，期望收到 PONG	✅ 通过
舵机单动	S1,500 → 读位置应为 500 左右	✅ 501

舵机双控	S1,400 / S2,600 → 分别读取	✅ 400 / 600
广播控制	S254,500 → 两个舵机同时动	✅ 通过
改 ID	ID1,2 → 再读 ID1 无应答，ID2 有应答	✅ 通过
信号自测	LOOP → 期望收到 0x55	✅ 通过
跟随	人员横向移动，观察舵机是否跟随	✅ 跟随 109 单位
丢目标处理	人员离开画面，舵机应回中位	✅ 回中

6.4 跟随实测统计

统计项	第一次测试	第二次测试
总处理帧数	4121 帧	3509 帧
成功检出	4121 帧（100%）	3509 帧（100%）
目标位置范围	x = 410 ~ 971	x = 346 ~ 1055
舵机跟随范围	465 ~ 552	454 ~ 563
舵机摆动幅度	87 单位（约 21°）	109 单位（约 26°）

说明：第二次测试时摄像头位置经过调整，视野更开阔，因此目标移动范围更大，舵机跟随幅度也更大。

七、成果与量化数据

7.1 技术成果

类别	成果
嵌入式固件	裸机固件 4.5KB，手写启动文件/中断向量表/链接脚本，无第三方库
协议实现	完整实现 LX-16A 二进制协议（位置/ID/装载/读回）
视觉系统	YOLOv8n + TensorRT FP16，37 FPS 实时检测
控制算法	偏移映射 + 指数平滑 + 安全限位，100% 检测率
工程工具	FlyMcu 自动烧录、三个诊断命令、系统化排查清单
系统集成	跨平台（Windows/Linux）、跨语言（C/Python）全链路打通

7.2 代码与文档统计

项目	数量	说明
源码文件	9 个	5 个 STM32 相关 + 4 个 Python
代码行数	约 1500 行	C 约 700 行，Python 约 800 行
文档篇数	8 篇	架构/接线/固件/协议/视觉/跟随/踩坑/性能
文档字数	约 42 KB	包含表格、代码块、数据
Git 提交	3 次	初始版本 + 报告 + 清理
项目报告	7 页	本 PDF

7.3 交付物清单

- GitHub 开源仓库：github.com/ZELDA-LINK-F/edge-ai-gimbal
- 可运行固件：STM32 裸机固件（含源码、链接脚本、构建脚本）
- 可运行应用：Jetson 视觉与跟随程序（camera/detect/servo/follow）
- 自动化工具：FlyMcu 自动烧录脚本
- 完整文档：8 篇技术文档 + 项目总结报告

- 可复现流程：从零开始按文档操作可完整复现整个系统

八、学习收获与反思

8.1 技术能力提升

方向	具体收获	验证方式
嵌入式底层	寄存器级编程、启动流程、中断向量表、链接脚本、时钟树配置	从零写出可运行的 4.5KB 固件
协议实现	二进制协议设计、帧格式、校验和、边界处理	LX-16A 协议完整实现并被舵机接受
边缘 AI	模型部署、TensorRT 加速、性能瓶颈定位	37 FPS 实时检测
控制算法	线性映射、指数平滑、安全限位设计	100% 检测率、±1 定位精度
系统集成	跨平台、跨语言联合调试	Jetson → STM32 → 舵机全链路
工程化	自动化脚本、诊断工具、可复现构建	一条命令烧录、三个诊断命令

8.2 工程方法反思

这次项目最有价值的不是「做出来了」，而是「怎么解决的」。以下是我总结的工程方法：

8.2.1 协议实现要先读文档再写代码

本次最大的教训。凭记忆写协议导致多传 2 字节，舵机静默丢弃，排查近 1 小时。正确做法：先完整读官方文档，写下帧格式，用官方例子做单元测试，确认无误再集成。

8.2.2 自制工具反复失败要换成熟工具

自制串口烧录器在「写内存」步骤反复失败，试了 20 多种组合都没解决。最后改用野火推荐的 FlyMcu，一次成功。教训：当自制工具在同一环节反复失败时，先用成熟工具验证硬件没问题，不要在错误方向越挖越深。

8.2.3 GUI 自动化失败先截图

FlyMcu 自动化时，三种技术方案都失败，最后截图才发现窗口被遮挡。教训：GUI 自动化失败时，先确认屏幕实际状态，不要凭空推测。

8.2.4 诊断工具的价值

固件里加入 LOOP / SCAN / BAUD 三个诊断命令，把原本需要示波器的排查变成了几秒钟的串口交互。教训：花 10% 的时间做诊断工具，能节省 50% 的调试时间。

8.2.5 自动化的投入产出比

花约 2 小时开发 FlyMcu 自动烧录脚本。看似不值，但当天迭代了十余次固件，每次都节省 1 分钟手动操作，而且避免了操作失误。教训：重复性工作值得自动化。

8.3 不足与改进

不足	影响	改进方向
协议实现前未完整读文档	浪费 1 小时排查	建立「先读文档—写测试—再集成」流程
机械结构未完成	当前为逻辑验证，非真正云台	完成摄像头固定结构
缺少自动化回归测试	每次改动需手动验证	编写 pytest 测试用例
文档写作在开发后	部分细节已模糊	开发过程中同步记录

九、后续规划

阶段	内容	技术要点	预期产出
近期	摄像头装至舵机	机械结构设计、FPC 走线	真正的云台跟随
近期	增加俯仰舵机（ID2）	二维控制、双舵机协同	支持上下注视
中期	接入视觉大模型	Ollama、qwen2.5vl:3b	可回答「画面里是什么」
中期	接入语音控制	faster-whisper、指令解析	语音控制云台
中期	人脸检测与跟随	人脸检测模型、多目标跟踪	更精细的追踪
长期	性能优化	INT8 量化、模型剪枝	更高帧率或更低功耗
长期	功耗测量与优化	功率计、动态频率调节	完整的边缘性能报告
长期	产品化	外壳、电源管理、稳定性	可长期运行的成品

9.1 技术演进路线

```
阶段一（已完成）：单目标水平跟随
    ↓
阶段二：二维云台（水平 + 俯仰）
    ↓
阶段三：多模态交互（视觉 + 语音）
    ↓
阶段四：智能行为（目标识别 + 决策）
    ↓
阶段五：产品化（结构、电源、稳定性）
```

本报告完整记录了项目的需求分析、技术选型、系统设计、实现过程、测试验证与经验总结。

项目代码与文档已开源于 GitHub：github.com/ZELDA-LINK-F/edge-ai-gimbal

— 报告完 —

附录 A：引脚分配总表

引脚	功能	方向	连接对象	备注
PA2	USART2_TX	输出	舵机总线	推挽发送 / 高阻让出
PA3	USART2_RX	输入	舵机总线	上拉 + 接收
PA9	USART1_TX	输出	CH340	与 PC/Jetson 通信
PA10	USART1_RX	输入	CH340	与 PC/Jetson 通信
GND	电源地	-	面包板负极轨	共地关键点
5V	电源输入	输入	USB	板子供电

附录 B：命令速查表

B.1 固件命令

命令	功能	示例
PING	心跳检测	→ PONG
S<id>,<pos>	舵机运动	S1,500
S<id>,<pos>,<ms>	定时运动	S1,500,1000
S254,<pos>	广播运动	S254,500
R<id>	读位置	R1
ID<old>,<new>	改 ID	ID1,2
LD<id>,<0|1>	卸载/装载	LD1,1
SCAN	扫描 ID 1~20	
LOOP	回环自测	
BAUD	波特率扫描	
T	发送测试帧	

B.2 Python API

方法	功能	示例
ServoBus()	创建总线对象（自动识别串口）	bus = ServoBus()

bus.scan()	扫描在线舵机	→ [(1, 501), (2, 498)]
bus.move(id, pos, ms)	舵机运动（受安全限位）	bus.move(1, 500, 1000)
bus.move(..., bypass=True)	绕过限位	bus.move(1, 300, bypass=True)
bus.read_pos(id)	读位置	→ 501
bus.center()	两个舵机回中位	bus.center()
bus.clamp(id, pos)	限位计算	→ (500, False)

附录 C：参考文献

[1] NVIDIA. Jetson Orin Nano Developer Kit User Guide. 2024.

[2] STMicroelectronics. STM32F103xx Reference Manual (RM0008). 2021.

[3] STMicroelectronics. STM32F10xxx Flash memory microcontrollers Programming manual (PM0068). 2015.

[4] STMicroelectronics. USART protocol used in the STM32 bootloader (AN3155). 2021.

[5] 野火. STM32 库开发实战指南——基于野火指南者开发板. 2021.

[6] 幻尔科技. LX-16A 总线舵机通信协议. 2020.

[7] Ultralytics. YOLOv8 Documentation. 2024.

[8] NVIDIA. TensorRT Developer Guide. 2024.

[9] NVIDIA. Accelerated GStreamer User Guide. 2024.

[10] 野火. STM32F103 指南者开发板硬件规格书. 2021.